

# Practical 5

```
In [11]: import numpy as np
import random
from collections import defaultdict

# =====
# Environment Parameters
# =====
GRID_SIZE = 4
ACTIONS = ['U', 'D', 'L', 'R']
TERMINAL_STATES = [(0, 0), (3, 3)]
START_STATE = (0, 1)

GAMMA = 1.0
EPISODES = 5000
EPSILON = 0.1

# =====
# Environment Step Function
# =====
def step(state, action):
    i, j = state

    # Terminal state
    if state in TERMINAL_STATES:
        return state, 0, True

    if action == 'U':
        i = max(i - 1, 0)
    elif action == 'D':
        i = min(i + 1, GRID_SIZE - 1)
    elif action == 'L':
        j = max(j - 1, 0)
    elif action == 'R':
        j = min(j + 1, GRID_SIZE - 1)

    next_state = (i, j)
    reward = -1
    done = next_state in TERMINAL_STATES

    return next_state, reward, done

# =====
# Policies
# =====
# Target policy (greedy)
def target_policy(Q, state):
    return np.argmax(Q[state])

# Behavior policy (epsilon-greedy)
def behavior_policy(Q, state):
    if random.random() < EPSILON:
        return random.randint(0, len(ACTIONS) - 1)
    return np.argmax(Q[state])

# =====
# Initialization
# =====
Q = defaultdict(lambda: np.zeros(len(ACTIONS)))
C = defaultdict(lambda: np.zeros(len(ACTIONS)))

# =====
# Off-Policy MC Control
# =====
for episode in range(EPISODES):

    episode_data = []
```

```

state = START_STATE

# Generate episode using behavior policy
while True:
    action_idx = behavior_policy(Q, state)
    action = ACTIONS[action_idx]

    next_state, reward, done = step(state, action)

    episode_data.append((state, action_idx, reward))
    state = next_state

    if done:
        break

# =====
# Backward Update
# =====
G = 0
W = 1

for t in reversed(range(len(episode_data))):
    state_t, action_t, reward_t = episode_data[t]

    G = GAMMA * G + reward_t

    C[state_t][action_t] += W
    Q[state_t][action_t] += (W / C[state_t][action_t]) * (G - Q[state_t][action_t])

    # Stop if action not greedy (importance sampling cut-off)
    if action_t != target_policy(Q, state_t):
        break

    # Update weight
    prob_behavior = (1 - EPSILON + EPSILON / len(ACTIONS))
    W = W * (1 / prob_behavior)

# =====
# Output: Q-values
# =====
print("\nState-Action Value Function (Q):\n")
for i in range(GRID_SIZE):
    for j in range(GRID_SIZE):
        if (i, j) in TERMINAL_STATES:
            print(f"State {(i,j)}: Terminal")
        else:
            print(f"State {(i,j)}: {Q[(i,j)]}")
    print()

# =====
# Output: Optimal Policy
# =====
print("\nOptimal Policy:\n")

policy_symbols = ['↑', '↓', '←', '→']

for i in range(GRID_SIZE):
    row = []
    for j in range(GRID_SIZE):
        if (i, j) in TERMINAL_STATES:
            row.append(' T ')
        else:
            best_action = np.argmax(Q[(i, j)])
            row.append(f" {policy_symbols[best_action]} ")
    print(" ".join(row))

print("\nTraining completed.")

```

State-Action Value Function (Q):

State (0, 0): Terminal  
State (0, 1): [ 0. 0. -1. 0.]  
State (0, 2): [0. 0. 0. 0.]  
State (0, 3): [0. 0. 0. 0.]  
  
State (1, 0): [-1. 0. 0. 0.]  
State (1, 1): [0. 0. 0. 0.]  
State (1, 2): [0. 0. 0. 0.]  
State (1, 3): [0. 0. 0. 0.]  
  
State (2, 0): [0. 0. 0. 0.]  
State (2, 1): [0. 0. 0. 0.]  
State (2, 2): [0. 0. 0. 0.]  
State (2, 3): [ 0. -1. 0. 0.]  
  
State (3, 0): [0. 0. 0. 0.]  
State (3, 1): [0. 0. 0. 0.]  
State (3, 2): [0. 0. 0. 0.]  
State (3, 3): Terminal

Optimal Policy:

|   |   |   |   |
|---|---|---|---|
| T | ↑ | ↑ | ↑ |
| ↓ | ↑ | ↑ | ↑ |
| ↑ | ↑ | ↑ | ↑ |
| ↑ | ↑ | ↑ | T |

Training completed.